

Modül 05 — Cevap Anahtarı

Veri işleme: dplyr ve tidyr

Prof. Dr. Hakan Mehmetcik

2026-09-11

Bu belge 05_modul_alistirmalar.qmd dosyasındaki 20 sorunun doğru cevaplarını ve gerekçelerini içerir. Gerekçeler yalnızca “doğru cevap bu” demekle kalmaz — **neden** doğru olduğunu da açıklar, çünkü sınavda sizden istenen de budur.

Bölüm A — `select()`, `filter()`, `arrange()`

- Doğru cevap: a)** `select()` **sütun** seçer, satırlara dokunmaz. Belirtilen sütunlar dışındaki tüm sütunlar sonuçtan çıkarılır; satır sayısı değişmez.
- Doğru cevap: b)** Virgülle ayrılan koşullar `filter()` içinde **VE** (mantıksal `&`) olarak birleşir. Bir gözlemin `continent` değeri aynı anda hem "Europe" hem "Asia" olamayacağı için bu iki koşulu **birlikte** sağlayan hiçbir satır yoktur. İstenen büyük olasılıkla `|` (VEYA) idi: `continent == "Europe" | continent == "Asia"`.
- Doğru cevap: b)** `arrange()`'in varsayılan sıralama yönü **artandır**. En düşük değerden başlar. Azalan sıralama için `desc()` eklenmelidir: `arrange(desc(gdpPercap))`.
- Doğru cevap: b)** `|` ile zincirlenen eşitlik koşulları ve `%in%` ile üye olma testi, sonlu bir küme için aynı satırları döndürür. `%in%` özellikle 3 veya daha fazla seçenek olduğunda daha kısa ve okunaklıdır.
- Doğru cevap: b)** `filter()` ve `arrange()` yalnızca **satırları** etkiler (hangi satırların kalacağı ve hangi sırada olacağı); sütun sayısını ve içeriğini yalnızca `select()` belirler. Zincirde `select(country, lifeExp)` çağrıldığı için sonuç 2 sütunludur.

Bölüm B — mutate(), group_by() + summarize()

6. Doğru cevap: b) mutate(), var olan sütunlardan yeni bir sütun **hesaplar/türetir**. rename() ise hiçbir hesaplama yapmaz, yalnızca bir sütunun **adını** değiştirir — değerleri olduğu gibi kalır.

7. Doğru cevap: b) Bu soru, mutate()'in summarise()'den farkını sınar: mutate() satır sayısını **korur**; sum(pop) burada gruplanmamış tüm satırların (yani dünyanın tüm ülkelerinin) nüfus toplamını alır ve bu **tek bir sayıyı** her satıra aynen dağıtır. Kod “her ülkenin kendi kıtasındaki payı” değil, “her ülkenin dünya nüfusundaki payı”nı hesaplar — eğer amaç kıta içi pay olsaydı, group_by(continent) eklenmesi gerekirdi (bkz. ders notu, group_by() bölümü).

8. Doğru cevap: b) group_by(continent), veriyi kıtaya göre 5 gruba ayırır (gapminder’da 5 farklı kıta vardır); summarise() her grubu **tek bir satıra** indirger. Sonuç: 5 satır.

9. Doğru cevap: b) group_by() tek başına veriyi özetlemez; yalnızca “bundan sonraki işlemler grup grup uygulansın” diye bir işaret bırakır. summarise() çağrılmadığı sürece satır sayısı değişmez — arrange() gruplanmış veriyi olduğu gibi sıralar.

10. Doğru cevap: b) Zincir sırasıyla: 2007 yılına ve Asya kıtasına filtrele, nüfusu 10 milyondan fazla olanları tut, yaşam beklentisine göre azalan sırada diz, yalnızca ülke adı ve yaşam beklentisini göster, ilk 3’ü al. Bu tam olarak (b) şıkkındaki soruya cevap verir.

Bölüm C — Düzenli (Tidy) Veri, pivot_longer(), pivot_wider()

11. Doğru cevap: c) Tidy verinin üç kuralı sütun **sayısıyla** değil, değişken/gözlem/değer ile sütun/satır/hücre arasındaki **eşleşmeyle** ilgilidir. Bir tablonun kaç sütunu olduğu tidy’liğini belirlemez.

12. Doğru cevap: b) “Yıl”, tek bir değişkendir; ama geniş formatta bu değişkenin farklı **değerleri** (1952, 1957, ...) ayrı **sütun adları** olarak kodlanmıştır. Bu, “her değişken kendi sütununu oluşturur” kuralının ihlalidir — çünkü aslında tek bir değişken 12 sütuna bölünmüştür.

13. Doğru cevap: a) names_to, dönüştürülecek eski **sütun adlarının** (burada yıllar) taşınacağı yeni sütunun adını alır. values_to, o sütunlardaki **hücre içeriğinin** (burada yaşam beklentisi sayıları) taşınacağı sütunun adını alır. İkisi ters yazılırsa tam olarak tarif edilen karışıklık ortaya çıkar.

14. Doğru cevap: a) pivot_wider(), names_from sütunundaki **değerleri** yeni sütun **adlarına**, values_from sütunundaki değerleri de bu yeni sütunların **içeriğine** dönüştürür. Bu, pivot_longer()'ın tam tersi yönde çalışır.

15. Doğru cevap: b) Hedef, yılların sütun olduğu bir tablo olduğuna göre bu **geniş** bir formattır ve `pivot_wider()` gerekir. Genel ilke: analiz/filtreleme için uzun format, okuma/sunma için geniş format tercih edilir.

Bölüm D — Birleştirmeler (Joins)

16. Doğru cevap: c) `uyelikler` tablosundaki her satır (bir ülke-örgüt eşleşmesi) `orgutler` tablosunda **tam olarak bir** eşleşme bulur (her `orgut_kodu` `orgutler`'de yalnızca bir kez geçer). Bu durumda `inner_join()` satır sayısını değiştirmez, yalnızca yeni sütunlar ekler: sonuç `uyelikler`'in satır sayısı olan 60'tır.

17. Doğru cevap: b) `left_join(x, y)`, `x`'teki (burada `gm2007`) **tüm satırları** korur. `y`'de (`uyelikler`) eşleşme bulunamayan satırlar için yeni eklenen sütunlar `NA` ile doldurulur — satır tablodan **atılmaz**.

18. Doğru cevap: b) `semi_join()` ve `inner_join()` **hangi satırların kalacağı** konusunda aynı sonucu verir, ama `semi_join()` `y`'den **hiçbir sütun eklemes** — yalnızca bir filtre gibi davranır. `inner_join()` ise eşleşen satırları tutarken `y`'nin sütunlarını da `x`'e ekler.

19. Doğru cevap: b) `anti_join()`, `y`'de karşılığı olmayan `x` satırlarını göstererek veri setinizin **kapsamındaki boşlukları** ortaya çıkarır — ör. bir referans listesinde olup elinizdeki veride karşılığı bulunmayan gözlemleri hızlıca tespit etmek için idealdir.

20. Doğru cevap: b) `by = c("sol_sütun" = "sağ_sütun")` sözdizimi, iki tablodaki **aynı bilgiyi tutan ama farklı adlandırılmış** sütunları birbirine eşler. `gapminder$country` ile `uyelikler$ulke` aynı şeyi (ülke adı) tutar ama farklı adlandırılmıştır; bu sözdizimi olmadan `join` fonksiyonu bu iki sütunu otomatik olarak eşleştiremez.